Electronics and
Computer Science

UNIVERSITY OF
Southampton

4. RISC-V Load, Store, and Branch Instructions

Load-Store Architecture

- RISC-V is a Load-Store (or Register-Register) architecture
  - as are MIPS, ARM (but this is not a defining characteristic of RISC)
- Load and Store are the only ways to access memory
- Alternatives include:
  - Register-Memory, e.g. x86, Z80

```
add r1, [mem2]
```

  - Memory-Memory, e.g. VAX

```
add [mem2], [mem2]
```

Core Instruction Formats

	31 25	24 20	19 15	14 12	11 7	6 0
R	funct7	rs2	rs1	funct3	rd	opcode
I	imm[11:0]		rs1	funct3	rd	opcode
S	imm[11:5]	rs2	rs1	funct3	imm[4:0]	opcode
B	imm[12|10:5]	rs2	rs1	funct3	imm[4:1|11]	opcode
U	imm[31:12]				rd	opcode
J	imm[20|10:1|11|19:12]				rd	opcode

Regularity – Note how imm bits are shuffled to maintain regularity

Load

- lw (load word) is an I-type instruction
  - `lw rd, imm(rs1)`
- e.g.
  - `lw x5, 40(x6)`
- This means load the contents of the memory address given by the contents of register x6+40 into register x5

imm(11:0)	000000101000	funct3	010
		rd	00101
rs1	00110	opcode	0000011

- `000000101000_00110_010_00101_0000011`
- `0000_0010_1000_0011_0010_0010_1000_0011`
- `02832283`

Other load instructions

- lb – loads bottom 8 bits, sign extends bit 7 (funct3 000)
- lh – loads bottom 16 bits, sign extends bit 15 (funct3 001)
- lbu – loads bottom 8 bits, top 24 bits are 0 (funct3 100)
- lhu – loads bottom 16 bits, top 16 bits are 0 (funct3 101)
- opcode is 0000011 in all cases
- rv64i also has ld – load double (register file is 32 x 64 bits)

Store

- sw (store word) is an S-type instruction
  - `sw rs2, imm(rs1)`
- e.g.
  - `sw x5, 40(x6)`
- This means store the contents of register x5 into the memory address given by the contents of register x6+40

imm(11:5)	0000001	funct3	010
rs2	00101	imm(4:0)	01000
rs1	00110	opcode	0100011

- `0000001_00101_00110_010_01000_0100011`
- `0000_0010_0101_0011_0010_0100_0010_0011`
- `02932423`

Other load instructions

- sb – stores bottom 8 bits (funct3 000)
- sh – stores bottom 16 bits (funct3 001)
- opcode is 0100011 in all cases
- rv64i also has sd – store double

- Notice the regularity
  - rs1, rs2, rd are always in the same positions
    - imm is split to maintain this
  - funct3 codes are the same for word, half-word, byte

if statement

- C code for if

```
if (i==j)
   f = g + h;
else
   f = g - h;
```

  - Compare 2 variables (registers) and make a decision

Conditional Branch Instructions

- B-type (or SB-type) instructions
  - Variant of S, but bit 12 of imm added and bit 0 removed (and bit 11 moved); bit 0 is implicitly 0
- Example
  - `beq rs1, rs2, L1`
- where L1 is a label in the assembly code
- The contents of rs1 and rs2 are compared. If (for beq) they are equal, the value of L1 (offset) is added to the value in the program counter
  - `PC <= PC + (offset << 1)`
- So, offset must be signed – 2's complement

Other Branch Instructions

- bne – branch if rs1, rs2 not equal
- blt – branch if rs1<rs2
- bge – branch if rs1>= rs2
- bltu, bgeu – rs1, rs2 treated as unsigned numbers

- opcode for all 6 branch instructions is 1100011
- funct3 is different for each

If statement

```
if (i==j) f = g + h; else f = g - h;
```

- Becomes (say):

```
        bne x22, x23, ELSE
        add x19, x20, x21
        beq x0, x0, EXIT
ELSE: sub x19, x20, x21
EXIT:
```

- Note `beq x0, x0, EXIT` is an unconditional jump. Why?
  - A compiler would (probably) not do this.

loops and other constructs

- while, for loops can be compiled in a similar way
  - bne when exit condition is met
  - jump (beq x0, x0, LOOP) to continue the loop
- switch/case is a bit more complicated
  - simplest solution is to chain if statements
  - more efficient to create a branch table and use an indirect jump – see later
- Note that all ISAs have similar instructions

Flags

- RISC V branch instructions depend on the comparison of rs1 and rs2
- ARM takes a different approach (as does MIPS)
  - 4 flags, set or not set as a result of the previous operation

V	oVerflow	Signed overflow
C	Carry	Unsigned carry
Z	Zero	Zero
N	Negative	Most significant bit is 1

  - BGE tests N and V flags
- RISC-V only has Z flag – simplifies pipelining

slt Instruction

- MIPS has flags but supports another mechanism
- slt – set less than
  - `slt rd, rs1, rs2`
- if rs1<rs2, rd is set to 1, else 0
- Then use beq or bne to test that register
- slti, sltu, sltiu – immediate and unsigned variants
- RISC-V also has slt, etc. (R- and I-types)